

Análisis de Viabilidad y Arquitectura: ScrapSAE Chrome Extension

Proyecto: ScrapSAE.Extension *Rama propuesta:* feature/chrome-extension
Fecha: Marzo 2026 *Basado en análisis del repositorio:* oscarfg78/ScrapSAE

1. Resumen Ejecutivo

El proyecto **ScrapSAE Chrome Extension** busca evolucionar la herramienta actual de scraping de escritorio hacia una extensión de navegador basada en **Manifest V3 (MV3)**. Esta transición permitirá a los usuarios ejecutar escaneos directamente desde la pestaña activa del navegador, guardar configuraciones de extracción personalizadas por usuario autenticado, exportar resultados a Excel con layouts configurables y sentar las bases para un modelo de monetización SaaS profesional.

Tras analizar en profundidad el repositorio actual y las capacidades de las extensiones de Chrome modernas, se concluye que el proyecto es **técnicamente viable** con una arquitectura híbrida (Extensión + Backend API existente). La mayor parte de la lógica de negocio desarrollada en C# puede reutilizarse directamente, lo que reduce significativamente el tiempo de desarrollo.

2. Análisis del Estado Actual del Proyecto

El repositorio `ScrapSAE` está estructurado como una solución .NET con arquitectura limpia y separación clara de responsabilidades. Los proyectos relevantes para la extensión son los siguientes:

Proyecto	Tecnología	Rol Actual	Rol en la Extensión
ScrapSAE.Core	C#	Entidades, DTOs e interfaces	Reutilizable íntegramente como contrato de datos
ScrapSAE.Api	ASP.NET Core	API REST + Playwright	Reutilizable y ampliable como backend de la extensión
ScrapSAE.Infrastructure	C#	Playwright, IA, exportación CSV	La lógica de IA y datos se reutiliza; Playwright se reemplaza
ScrapSAE.Desktop	WPF	UI de escritorio	No se reutiliza en la extensión; sirve de referencia de UX
ScrapSAE.Worker	.NET Worker	Servicio de fondo en Windows	No se toca ; sigue funcionando para la integración con SAE

El modelo `SiteSelectors` en `DTOs.cs` es particularmente valioso: ya define más de 30 propiedades de configuración de layout (selectores de lista, detalle, paginación, scroll infinito, adjuntos, stock, variantes), lo que significa que el sistema de layouts configurables ya existe a nivel de modelo y solo necesita exponerse en la UI de la extensión.

La lógica de evasión de detección también está bien desarrollada en el `stealth_script.js` actual, que incluye técnicas como eliminación de `navigator.webdriver`, simulación de plugins, ruido en Canvas y modificación de WebGL. Este script puede reutilizarse directamente en la extensión.

3. Viabilidad Técnica por Funcionalidad

3.1. Extracción desde la Pestaña Activa

Viabilidad: Alta. Las extensiones de Chrome pueden inyectar *Content Scripts* en cualquier página activa con el permiso `activeTab`. Estos scripts tienen acceso completo al DOM de la página y pueden extraer cualquier elemento usando los

selectores CSS ya definidos en `SiteSelectors`. La comunicación entre el Content Script y el resto de la extensión se realiza mediante el sistema de mensajería de Chrome (`chrome.runtime.sendMessage`). [1] [2]

3.2. Simulación de Comportamiento Humano

Viabilidad: Alta. Esta es una de las ventajas más importantes de la extensión frente a Playwright: al ejecutarse dentro de un navegador real con el perfil del usuario, la extensión **no activa los detectores de automatización** que buscan señales de `navigator.webdriver` o patrones de tráfico de red inusuales. Adicionalmente, el Content Script puede implementar:

- Pausas aleatorias entre acciones (mínimo 1 segundo, con variación aleatoria).
- Movimientos de ratón simulados con curvas de Bézier antes de hacer clic en elementos.
- Scroll progresivo y no lineal para simular lectura.
- El `stealth_script.js` existente puede inyectarse como `content_scripts` en el `manifest.json` para mantener la evasión de fingerprinting. [3]

3.3. Exportación a Excel con Layout Configurable

Viabilidad: Alta. La biblioteca **SheetJS** (`xlsx`) es compatible con extensiones de Chrome MV3 tanto en Content Scripts como en el Background Service Worker. Permite generar archivos `.xlsx` con columnas dinámicas, múltiples hojas y estilos. Para la descarga, se usa `chrome.downloads.download()` con una URL de datos en Base64, ya que `URL.createObjectURL` no está disponible en Service Workers. [4]

El flujo sería: el backend procesa los datos y los devuelve como JSON estructurado → el Background Service Worker usa SheetJS para construir el workbook con el layout del usuario → se descarga el archivo `.xlsx` automáticamente.

3.4. Autenticación y Configuración por Usuario

Viabilidad: Alta. Supabase Auth es compatible con extensiones de Chrome. El flujo de autenticación abre una nueva pestaña para el login (Google OAuth o email/contraseña), y una vez autenticado, el token de sesión se almacena en `chrome.storage.local` para persistir entre sesiones. El Background Service Worker

monitorea los cambios de URL para detectar el callback de OAuth y completar el flujo. [5]

La configuración de layouts y selectores se almacena en Supabase (tablas `user_layouts` y `config_sites`) asociada al `user_id` del usuario autenticado, permitiendo que el usuario acceda a sus configuraciones desde cualquier dispositivo.

3.5. Historial de Ejecuciones

Viabilidad: Alta. La tabla `execution_reports` ya existe en el esquema de Supabase. Solo se requiere agregar la columna `user_id` para asociar cada ejecución al usuario que la realizó. La extensión puede mostrar el historial en un panel lateral (Side Panel API de Chrome MV3).

3.6. Múltiples Layouts

Viabilidad: Alta. Se crea una tabla `user_layouts` en Supabase con la estructura del `SiteSelectors` serializado como JSONB, asociado al `user_id` y con un nombre descriptivo. El usuario puede crear, editar, duplicar y eliminar layouts desde la UI de la extensión.

4. Restricciones y Consideraciones Importantes

Existen algunas restricciones de MV3 que deben tenerse en cuenta durante el desarrollo:

Ciclo de vida del Service Worker. El Background Service Worker de MV3 no es persistente: Chrome puede terminarlo en cualquier momento cuando está inactivo. Todo el estado que deba sobrevivir entre eventos debe guardarse en `chrome.storage.local` o `chrome.storage.session`. Esto implica que el progreso de un scraping largo debe guardarse incrementalmente.

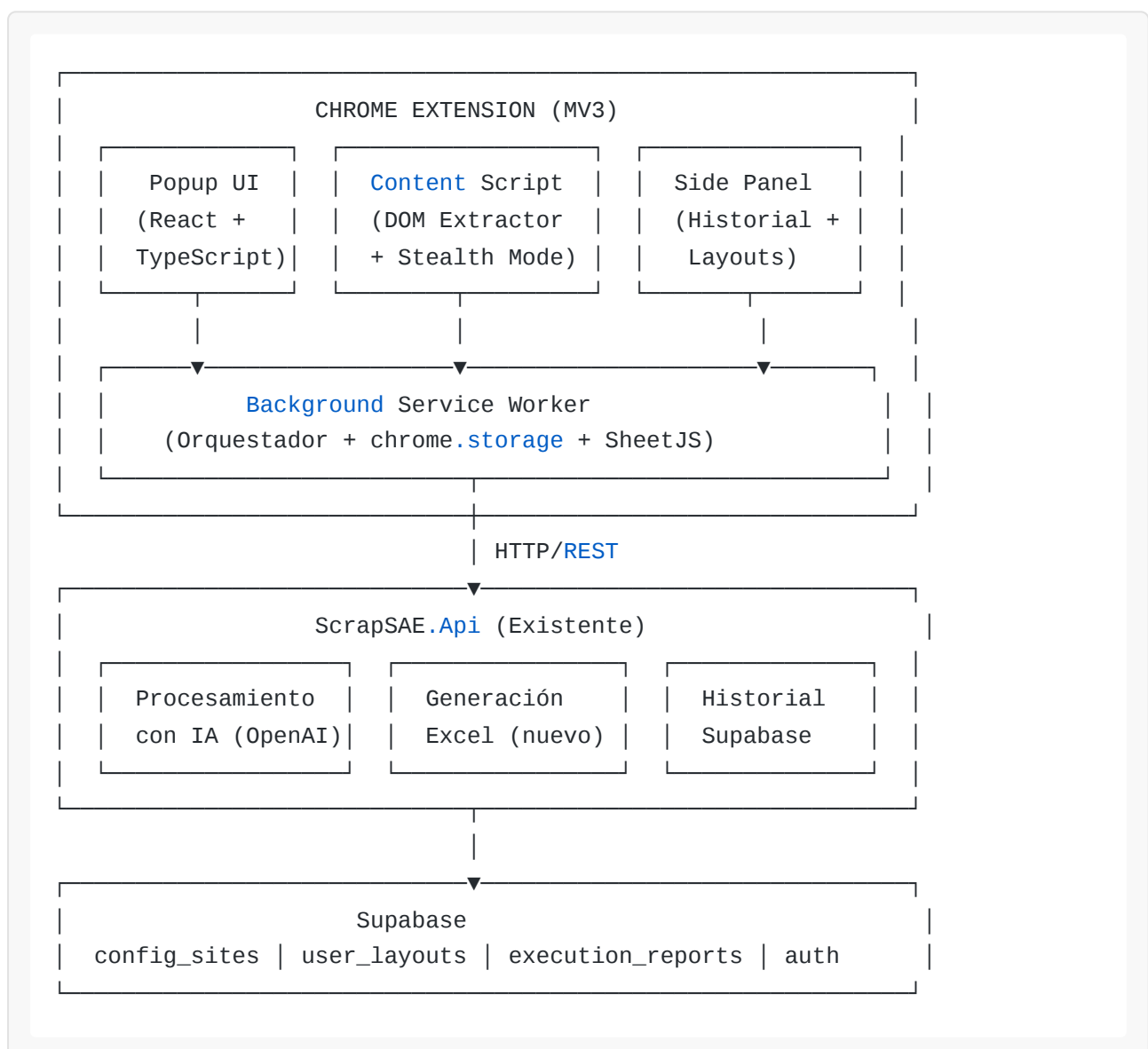
Content Security Policy (CSP). Las extensiones MV3 tienen una CSP más estricta. No se puede ejecutar código JavaScript evaluado dinámicamente (`eval()`). Todo el código debe estar empaquetado en los archivos de la extensión. El `stealth_script.js` actual es compatible con esta restricción.

Permisos de Host. Para inyectar el Content Script en páginas de proveedores específicos (como Festo), la extensión necesita declarar los permisos de host en el `manifest.json`. Para una herramienta genérica, se puede solicitar el permiso `activeTab` que solo activa el script cuando el usuario hace clic en el botón de la extensión, lo cual es menos invasivo y más fácil de aprobar en la Chrome Web Store.

Manifest V2 ya no es soportado. Chrome 138 y versiones posteriores no soportan extensiones MV2. El desarrollo debe hacerse directamente en MV3. [4]

5. Arquitectura Propuesta

Se propone una arquitectura híbrida de tres capas que maximiza la reutilización del código existente:



5.1. Componentes de la Extensión

Popup UI es la ventana principal que aparece al hacer clic en el ícono de la extensión. Desarrollada en React + TypeScript, permite al usuario iniciar sesión, seleccionar el layout de extracción deseado, configurar opciones (modo simulación, límite de productos) e iniciar el scraping. Muestra el progreso en tiempo real mediante mensajes del Service Worker.

Content Script es el componente más crítico. Se inyecta en la pestaña activa cuando el usuario inicia un scraping. Implementa el módulo de simulación humana, lee el DOM basándose en los selectores del layout seleccionado, maneja la paginación (clic en “siguiente página” o scroll infinito) y envía los datos crudos al Background Service Worker en lotes.

Background Service Worker actúa como orquestador. Recibe los datos del Content Script, los envía al Backend API para procesamiento, recibe el JSON estructurado de respuesta, usa SheetJS para construir el archivo Excel con el layout del usuario y lo descarga. También gestiona el estado de la sesión de Supabase.

5.2. Nuevo Proyecto en la Solución

Se agregará el proyecto `ScrapSAE.Extension` a la solución `ScrapSAE.sln` como un proyecto de tipo Node.js/Vite. La estructura será:

```
ScrapSAE/
├─ src/
│   ├─ ScrapSAE.Api/           (existente, ampliado)
│   ├─ ScrapSAE.Core/         (existente, sin cambios)
│   ├─ ScrapSAE.Infrastructure/ (existente, sin cambios)
│   ├─ ScrapSAE.Desktop/      (existente, sin cambios)
│   ├─ ScrapSAE.Worker/       (existente, sin cambios)
│   └─ ScrapSAE.Extension/    ← NUEVO PROYECTO
│       ├─ src/
│           ├─ background/    (Service Worker)
│           ├─ content/       (Content Scripts)
│           ├─ popup/         (React UI)
│           ├─ sidepanel/     (Historial y Layouts)
│           └─ shared/        (DTOs compartidos en TS)
│       └─ public/
│           ├─ manifest.json
│           └─ stealth_script.js (reutilizado)
│       └─ package.json
│       └─ vite.config.ts
```

6. Stack Tecnológico de la Extensión

Componente	Tecnología	Justificación
Framework UI	React 18 + TypeScript	Consistente con el Dashboard existente
Build Tool	Vite + CRXJS	Especializado para extensiones Chrome con HMR
Estilos	TailwindCSS	Consistente con el resto del proyecto
Generación Excel	SheetJS (xlsx)	Compatible con MV3, soporta layouts dinámicos [4]
Autenticación	Supabase Auth	Ya usado en el proyecto; soporta extensiones [5]
Estado	chrome.storage.local	Persistencia nativa de extensiones
Comunicación	chrome.runtime.sendMessage	API estándar de mensajería de extensiones
Simulación	JavaScript nativo	Curvas de Bézier, setTimeout aleatorio

7. Estrategia de Monetización

7.1. Modelo de Precios

Plan	Precio	Límites	Características
Free	Gratis	50 productos/día, 1 layout	Exportación básica, historial 7 días
Pro	\$19 USD/mes	Ilimitado, layouts ilimitados	Exportación personalizada, historial completo, soporte
Enterprise	\$79 USD/mes	Multi-usuario, integración SAE	Todo Pro + Worker de escritorio + soporte dedicado

7.2. Infraestructura de Pagos

Se integrará **Stripe** como pasarela de pagos, ya que es la opción más robusta para SaaS con soporte para suscripciones recurrentes, pruebas gratuitas y webhooks. [6] El flujo completo es:

1. El usuario accede a la Landing Page y selecciona un plan.
2. Stripe Checkout gestiona el pago de forma segura.
3. Stripe envía un webhook `checkout.session.completed` al Backend API.
4. El Backend actualiza el campo `subscription_status` en la tabla `users` de Supabase.
5. La extensión consulta el estado al iniciar y desbloquea las funciones correspondientes.

7.3. Landing Page

La Landing Page será desarrollada como un proyecto separado (`ScrapSAE.Web`) usando **Next.js** o **Astro** para máximo rendimiento SEO. Las secciones clave serán:

- **Hero:** Propuesta de valor directa con video demo del scraping en acción.
- **Características:** Extracción indetectable, Excel a medida, historial en la nube.
- **Pricing:** Tabla comparativa de planes con botón de “Empezar Gratis”.
- **Testimonios:** Casos de uso reales (ej. distribuidores que usan Festo).
- **FAQ:** Preguntas sobre compatibilidad, seguridad y datos.
- **Footer:** Links a política de privacidad, términos de uso y soporte.

8. Plan de Proyecto Detallado

Preparación del Repositorio

Antes de iniciar el desarrollo, se deben realizar los siguientes pasos en Git:

```
# Crear la rama de trabajo
git checkout -b feature/chrome-extension

# Inicializar el proyecto de la extensión
cd src
npm create crxjs@latest ScrapSAE.Extension -- --template react-ts

# Agregar el proyecto a la solución (referencia en .sln)
# Se hace manualmente editando ScrapSAE.sln
```

Cronograma de Implementación

Fase	Duración	Entregables Clave
Fase 1: Estructura y Auth	2 semanas	Proyecto inicializado, login con Supabase funcional, manifest.json configurado
Fase 2: Motor de Extracción	2 semanas	Content Script extrayendo datos de Festo, simulación humana activa
Fase 3: Backend y Excel	2 semanas	Endpoints de procesamiento, generación de .xlsx con layout configurable
Fase 4: Layouts e Historial	2 semanas	UI de gestión de layouts, historial de ejecuciones en Side Panel
Fase 5: Monetización	2 semanas	Landing Page, Stripe integrado, publicación en Chrome Web Store

Tiempo total estimado: 10 semanas (2.5 meses) para un equipo de 1-2 desarrolladores.

9. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Cambios en el DOM de Festo	Alta	Alto	El sistema de layouts configurables permite actualizar selectores sin redeploy
Rechazo en Chrome Web Store	Media	Alto	Revisar políticas de scraping; enfocar en uso legítimo de datos propios
Service Worker terminado durante scraping largo	Media	Medio	Guardar progreso en <code>chrome.storage.session</code> cada N productos
Detección de bot por Festo	Baja	Alto	Simulación humana + uso del perfil real del usuario en el navegador

10. Conclusión

La extensión de Chrome para ScrapSAE es un proyecto **altamente viable** que puede desarrollarse en aproximadamente 10 semanas. Las ventajas clave sobre la versión de escritorio son:

- **Sin instalación de Playwright:** El navegador del usuario es el motor de scraping, lo que elimina la detección de automatización.
- **Reutilización masiva:** Los DTOs, la lógica de IA, el procesamiento de datos y la infraestructura de Supabase ya existen.
- **Escalabilidad SaaS:** La arquitectura propuesta está diseñada desde el inicio para soportar múltiples usuarios con configuraciones independientes y un modelo de monetización robusto.
- **No rompe lo existente:** Al trabajar en una rama separada y agregar un nuevo proyecto a la solución, el Worker de escritorio y la integración con SAE continúan funcionando sin cambios.

Referencias

- [1] Chrome for Developers. "Resuming the transition to Manifest V3".
<https://developer.chrome.com/blog/resuming-the-transition-to-mv3>
- [2] Chrome for Developers. "Content scripts".
<https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>
- [3] Bright Data. "Avoiding Bot Detection with Playwright Stealth".
<https://brightdata.com/blog/how-tos/avoid-bot-detection-with-playwright-stealth>
- [4] SheetJS. "Chrome and Chromium Extensions".
<https://docs.sheetjs.com/docs/demos/extensions/chromium/>
- [5] Tomas Pustelnik. "How to implement Supabase auth in a browser extension".
<https://pustelto.com/blog/supabase-auth/>
- [6] Dodo Payments. "How to Monetize a Chrome Extension in 2026".
<https://dodopayments.com/blogs/monetize-chrome-extension>